

Chapter 14. 품질 기준을 프로젝트마다 다르게 — 설정 가능한 Harness

이 챕터에서 배우는 것

- Gate 가중치가 왜 CLI 플래그가 아니라 `.env` 로 노출됐는지
- "죽은 파라미터"가 무엇이었고 어떻게 실제로 배선됐는지
- 관대한 파싱(값이 잘못돼도 전체를 죽이지 않는) 원칙이 여기서 어떻게 적용되는지

이런 분이 먼저 읽으면 좋습니다: 9장에서 본 Gate A의 가중 평균 공식($0.4 \times \text{TCR} + 0.6 \times \text{나머지}$)이 고정값처럼 보였는데, 실제로 조정할 방법이 있는지 궁금한 분.

14.1 발견 — 이미 있던 파라미터가 아무도 안 쓰고 있었다

`eval/monitor.py`의 `build_book_monitor()`는 처음부터 `enable_llm_judge` / `judge_model` 파라미터를 갖고 있었다. 그런데 실제로 확인해 보니, `new` · `draft` · `chat` · `build` · `research` · `plan` · `review` 7개 CLI 명령어 **어느 하나도 이 값을 실제로 넘기지 않았다** — 항상 기본값(꺼짐)만 쓰였다. 함수 시그니처에는 있지만 실질적으로 절대 커지지 않는, **죽은 파라미터**였다. Gate 가중치 (`gate_a_tcr_weight` 등, SDK가 이미 지원)도 마찬가지로 어디에도 노출돼 있지 않았다.

이 발견은 이 챕터 전체의 출발점이다 — "기능이 코드에 존재한다"는 것과 "그 기능이 실제로 도달 가능하다"는 것은 다른 문제다.

14.2 배선 — CLI 플래그로 LLM Judge를 옵트인시킨다

```

@click.option(
    "--enable-llm-judge", is_flag=True,
    help="일부 샘플에 LLM 채점(faithfulness 등)을 추가 적용 (기본 off - 비용/지연 증가)",
)
@click.option(
    "--judge-model", default=None,
    help="[--enable-llm-judge] 채점에 쓸 모델 (미지정 시 API 키 기반 자동 결정)",
)
def new(..., enable_llm_judge: bool, judge_model: Optional[str]) -> None:
    ...
    monitor = build_book_monitor(
        output_dir=str(project_dir / "eval_results"),
        enable_llm_judge=enable_llm_judge,
        judge_model=judge_model,
    )

```

새 판정 로직은 전혀 없다 — 이미 있던 파라미터를 CLI 플래그까지 실제로 연결했을 뿐이다. `draft_cmd.py` 도 같은 방식으로 배선했다. `enable_llm_judge` 가 기본 off인 이유는 명확하다 — Agent-Evaluator의 `LLMJudge` 는 OpenAI/Anthropic 모델만 지원한다(Ollama 연동이 없다, `grep` 으로 확인). 1장에서 다룬 "기본 provider는 Ollama, API 키 없이 동작"이라는 원칙과 이 옵션은 정면으로 부딪힌다 — 그래서 명시적으로 켜야만 API 키를 요구하는 경로로 들어간다.

14.3 Gate 가중치 — 왜 CLI 플래그가 아니라 `.env` 인가

```

_GATE_WEIGHT_ENV_VARS = (
    ("BOOK_FORGE_GATE_A_TCR_WEIGHT", "gate_a_tcr_weight"),
    ("BOOK_FORGE_GATE_C_TCR_WEIGHT", "gate_c_tcr_weight"),
    ("BOOK_FORGE_GATE_B_LOOP_WEIGHT", "gate_b_loop_weight"),
)

def _gate_weight_overrides() -> dict[str, float]:
    overrides: dict[str, float] = {}
    for env_name, kwarg_name in _GATE_WEIGHT_ENV_VARS:
        raw = os.environ.get(env_name)
        if raw is None:
            continue
        try:
            overrides[kwarg_name] = float(raw)
        except ValueError:

```

```
        continue
    return overrides
```

Gate 가중치는 매번 명령을 입력할 때마다 타이핑하는 값이 아니라 "이 환경에서는 항상 이 기준으로 판정한다"는 **저자별 고정 설정**에 가깝다고 판단해 `.env` 로 노출했다 — 1장에서 다룬 `LLM_PROVIDER` 선택과 정확히 같은 성격이다.

`build_book_monitor()` 가 이 값을 `**_gate_weight_overrides()` 로 `PerformanceMonitor` 에 그대로 펼쳐 넘긴다.

```
return PerformanceMonitor(
    output_dir=output_dir,
    enable_security_metrics=True,
    enable_hallucination_detection=False,
    enable_llm_judge=enable_llm_judge,
    judge_model=judge_model,
    judge_sample_rate=0.2,
    auto_save=True,
    auto_save_interval=5,
    *_gate_weight_overrides(),
)
```

14.4 관대한 파싱 — 오타 하나가 파이프라인을 죽이면 안 된다

`_gate_weight_overrides()` 의 `try/except ValueError: continue` 는 이 코드베이스에서 반복되는 원칙을 그대로 따른다 — 함수 docstring이 명시한다.

"값이 float으로 파싱 안 되면(오타 등) 그 항목만 조용히 무시한다 —

`alternative_suggester.py` 의 '관대한 파싱, 실패해도 결과는 낸다' 원칙과 동일."

`.env` 에 `BOOK_FORGE_GATE_A_TCR_WEIGHT=zero point four` 처럼 잘못 쓰더라도, 그 한 줄 때문에 `book-forge new` 전체가 예외로 죽지 않는다 — 그 항목만 무시하고 `PerformanceMonitor` 의 원래 기본값(`gate_a_tcr_weight=0.4`)으로 조용히 되돌아간다. 이는 4장 (§ 4.3)에서 다룬 "관대한 파싱"과 6장 (§ 6.2)에서 다룬 "안전한 폴백" 원칙이 설정 값 파싱에도 일관되게 적용된 사례다.

📄 **QA 관리자 TIP:** 이 설계는 "설정이 잘못돼도 조용히 기본값으로 넘어간다"는 뜻이기도 하다 — 오타를 낸 걸 사용자가 알아채지 못할 수 있다는 트레이드오프가 있다. Book-forge 전체가 "파이프라인을 절대 죽이지 않는다"는 방향을 일관되게 택하고 있다는 것을 이해하고 나면, 설정이 의도대로 반영됐는지 별도로 확인하는 습관 (`PerformanceMonitor` 인스턴스의 실제 속성값을 로그로 남기는 등)이 왜 필요한지 알 수 있다.

14.5 실측 — `.env` 값이 실제로 반영되는지 API 키 없이 확인 가능하다

Gate 가중치 조정은 API 키가 필요 없는 경로다 — `.env` 파일에 `BOOK_FORGE_GATE_A_TCR_WEIGHT=0.9` 를 쓰고, 실제 `load_config()` (python-dotenv) → `os.environ` → `PerformanceMonitor(gate_a_tcr_weight=0.9)` 전체 경로가 몽키패치 없이 실측으로 확인됐다 — `.env` 에 쓴 값이 실제로 `PerformanceMonitor` 인스턴스의 내부 속성에 그대로 반영됨을 직접 실행으로 검증할 수 있었다.

이 챕터의 핵심

- **파라미터가 시그니처에 있다고 실제로 도달 가능한 것은 아니다.**
`enable_llm_judge` 가 7개 CLI 명령 어디서도 실제로 안 쓰이던 "죽은 파라미터"였던 사례가 이를 보여준다.
- **설정을 CLI 플래그로 둘지 `.env` 로 둘지는 "매번 바뀌는가, 환경마다 고정인가"로 결정한다.** LLM Judge는 옵트인 플래그, Gate 가중치는 `.env` .
- **관대한 파싱은 설정 값에도 일관되게 적용된다.** 오타 하나가 전체 파이프라인을 죽이지 않는다.

참고 자료

- `src/book_forge/eval/monitor.py` — 전체

- `src/book_forge/cli/commands/new_cmd.py` · `draft_cmd.py` — 실제 플래그 배선
-

마지막 챕터는 지금까지 다룬 모든 메커니즘의 경계 밖 — Book-forge가 아직 하지 못하는 것을 정직하게 다룬다.